COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	October 7, 2026	**DATE SUBMITTED:**	October 9, 2026
NAME:	Marc Jhowell M. Bungag Ramelissa Melith V. Limpio	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 11.1 **Searching Techniques**

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Worst Case: $n^2 + n$

The arrays share no common values, so every search call fails and scans all of x.

Analysis

Bungag: In the worst-case scenario, this algorithm checks every element of array y to all elements of array x. If array x has m elements and array y has n elements, the total number of comparisons is mn + n, including the loop operations. Since both arrays have the same size (m = n), the formula becomes $n^2 + n$.

Limpio: In worst-case, there are no common values, so the loop runs n times and each search makes m comparisons: m x n total, or n^2 when m = n (n^2 + n counting the loop). The graph (110, 420, 930) gets steeper as n grows, so the growth is quadratic: $O(n^2)$

Specifications		
PC PART	**BUNGAG**	**LIMPIO**
Processor	Intel Core - i5 12th Gen	Intel Core - i7 13th Gen
RAM	32GB	16GB
Storage	1TB	1TB
OS	Windows 11	Windows 11

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Theoretical Analysis: O(log n)

Bungag			
Input Size (N)	**Execution Speed**	**Screenshot**	**Observation(s)**
1,000	1µs	Time taken to search = 1 microseconds Student (648 49) needs vaccination.	The execution time is 1 microsecond; this indicates that the search performs quickly.
10,000	1µs	Time taken to search = 1 microseconds Student (648 49) needs vaccination.	The executions time remains at 1 microsecond despite the increase in input size
100,000	3µs	Time taken to search = 3 microseconds Student (648 49) needs vaccination.	The execution time jumps at m3 microseconds, even with the larger dataset the increase in run time is minimal.

Limpio			
Input Size (N)	**Execution Speed**	**Screenshot**	**Observation(s)**
1,000	0µs	Time taken to search = 0 microseconds Student (120 114) needs vaccination.	The execution time of 1,000 input size is 0 microseconds; this indicates that the operation is quick for such a small dataset.
10,000	1µs	Time taken to search = 1 microseconds Student (120 114) needs vaccination.	The execution time of 10,000 inputs is 1 microsecond. Even with the increase in the size, the search time increased slightly.
100,000	1µs	Time taken to search = 1 microseconds Student (120 114) needs vaccination.	The execution time with 100,000 input size is also 1 microsecond. This states that the operation handles larger datasets efficiently.

Table 11-1

Analysis

Bungag: The search algorithm demonstrates efficient performance as the input size increases. The time stays at 1 microsecond for 1,00 and 10,000 inputs, then increases to 3 microseconds for 100,000 inputs. It shows that the search algorithm can handle larger datasets with only a small increase in the execution time.

Limpio: Binary search halves the range each step, so it takes about $\log_2 N$ comparisons (~10, 13, 17). Input grows 100× but work barely changes, so timings stay close together. This matches theoretical analysis, O(log N), meaning binary search stays fast even for very large sorted lists.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Consider a program that returns true if the elements in an array are unique.

Theoretical Analysis

Worst Case (all unique): **O(n^2)**
Best case (A[0] == A[1]): **O(1)**
Space: **O(1)**

Experimental Analysis

n	count	n(n-1)/2	time(us)
1000	499500	499500	7037
2000	1999000	1999000	40919
4000	7998000	7998000	84398
8000	31996000	31996000	395847
16000	127992000	127992000	1108512

Analysis & Conclusion

Bungag: The experimental results show that the number of comparisons increases quadratically as the array size doubles. The execution time also increases as the input size increases, which means they are directly proportional.

Limpio: The count matches n(n−1)/2 at every size, confirming O(n^2). Doubling n makes the count (and the time) grow about 4 times, with small timing noise. It is only practical for small to medium arrays.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

Theoretical Analysis

	Recurrence	Time Complexity
rpower	T(n) = T(n−1) + c	O(n)
brpower	T(n) = T(n/2) + c	O(log n)

Experimental Analysis

n	rp mult	rp ns	brp mult	brp ns
1000	1000	99500	16	500
2000	2000	157100	17	500
4000	4000	266500	18	400
8000	8000	373100	19	400
16000	16000	1024500	20	500

Analysis & Conclusion

Bungag: The results show that rpower performs more multiplications as the input size increases, while bpower requires only a small number of multiplications. This states that bpower is more efficient, especially for large values of n.

Limpio: rpower does n multiplications (O(n)), while brpower does only $\log_2 n$ (16 to 20), so its time stays flat at ~500 ns. At n = 16,000, that's 16,000 multiplications versus 20, making brpower far more efficient.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.